5 - Conclusions

And that's it. As you may have noticed, even creating a simple window as the basis for Vulkan rendering operations can be quite involved, as it requires a significant amount of knowledge and code. However, once you've learned how to display a window on the screen, you will have a solid foundation for building any Vulkan application.

Although this tutorial may have presented some challenges, if you've made it through, you'll be pleased to know that you've already made significant progress in learning Vulkan. Indeed, it is often said that Vulkan has a steep learning curve, but I like to think of it more as a staircase with decreasingly smaller steps. By completing this first tutorial, you have climbed the first step and overcome the most significant obstacle. From this point onward, things can only get easier.

> If you set **Settings::vsync** to **true**, then the presentation mode selected will be FIFO. In that case, the title bar will display an FPS count that matches the refresh rate of the monitor. However, if **vsync** is set to **false**, the FPS count will be much higher, even if the selected presentation mode id Mailbox, which still requires the presentation manager to wait for the next v-sync before showing a new image on the screen. This is because the FPS counter implemented in this demo counts the frames created on the CPU timeline, rather than those actually displayed on the screen. Additional information will be provided in an upcoming tutorial, where we will also discuss frame buffering and latency.

Source code: LearnVulkan

References

[1] Vulkan API Specifications
[2] Vulkan Guide
[3] Vulkan Loader
[4] Vulkan Tools
[5] Vulkan Samples
[6] Sascha Willems on GitHub
